Invoice Validation Pipeline — Technical Specification

Nov 4, 2026 · @Buddhika Amarasinghe

1. Overview and scope

The pipeline takes each supplier invoice PDF from Kafka to exactly one ERP posting. An agent extracts the fields, a deterministic rule engine validates the date, total and line items, and anything uncertain goes to a human review UI. Two failures matter most: paying a supplier twice (a duplicate posting) and not paying one (a lost invoice). Every design choice below serves those two.

Goals

- **No double posting.** Exactly-once effect at the ERP, even though Kafka delivers at least once.
- **No lost invoice.** Every message reaches a terminal state: Posted, Rejected or Duplicate, each with an audit trail.
- **Uncertain means human.** Any low-confidence field or failed rule goes to the review queue, never straight to the ERP.
- **Month-end ready.** Handle 3× volume without breaching SLA.

Confirmed requirements

Area	Decision
UI scope	Exception review queue + review workspace + ops dashboard
Total	Σ(qty × unit price) + tax + charges = stated total, with a per-currency tolerance
Date	Not in the future · not older than 90 days · not earlier than the PO date · falls in an open ERP posting period
Line items	Each line maps to a PO line and an ERP item master record; price ±2% of PO, quantity exact (⩽ open PO quantity)
Controls	Maker-checker above LKR 1,000,000 equivalent · mandatory reason codes · SLA timers with escalation · access by legal entity

Area	Decision
Stack	React/Next.js UI · FastAPI services · SQLite for dev and pilot, Postgres in production (same schema via SQLAlchemy)
Integration	Kafka (Redis Streams noted as an alternative) · generic ERP adapter

Out of scope: payment runs, supplier onboarding, PO creation, OCR model training, and ERP-side approval workflows.

Assumption: you confirmed the defaults but didn't choose between the two SQLite options, so this spec uses option B, the recommended one. SQLite runs only where there is a single writer (local dev and a single-node pilot). Production uses Postgres, because the duplicate guard needs one shared, concurrent store.

2. Capacity estimates

The machine side is small: about 8 invoices a second at the month-end peak. The real constraints are the model provider's rate limits and reviewer headcount. All figures are planning estimates to confirm in the pilot.

Measure	Normal day	Month-end day	Basis
Invoices / day	20,000	60,000	Given (3×)
Average arrival rate	0.6 / s	1.7 / s	Arrivals over a 10-hour business window
Peak burst	3 / s	8 / s	5× average (batch drops from suppliers and EDI hubs)
Extraction latency	p50 8 s, p95 20 s	same	Assumes a 1–3 page PDF, one multimodal call
In-flight model calls at peak	~45	~120	Little's law: burst rate × p95 latency, with headroom
Tokens / invoice	~4k (3k in, 1k out)	same	Assumption; re-measure on a pilot sample
Tokens / day	~80M	~240M	Sets the provider tier and rate-limit request

Measure	Normal day	Month-end day	Basis
Exceptions to people (10%)	2,000	6,000	Target auto-pass rate 90% after tuning
Reviewer effort at 2.5 min each	~83 h ≈ 11 FTE	~250 h ≈ 31 FTE	Biggest cost lever; see note below
PDF storage	~4 GB	~12 GB	~200 KB per PDF; ~1.5 TB a year, kept 7 years
State DB rows	~250k	~750k	~12 rows per invoice (header, lines, events, rule results)

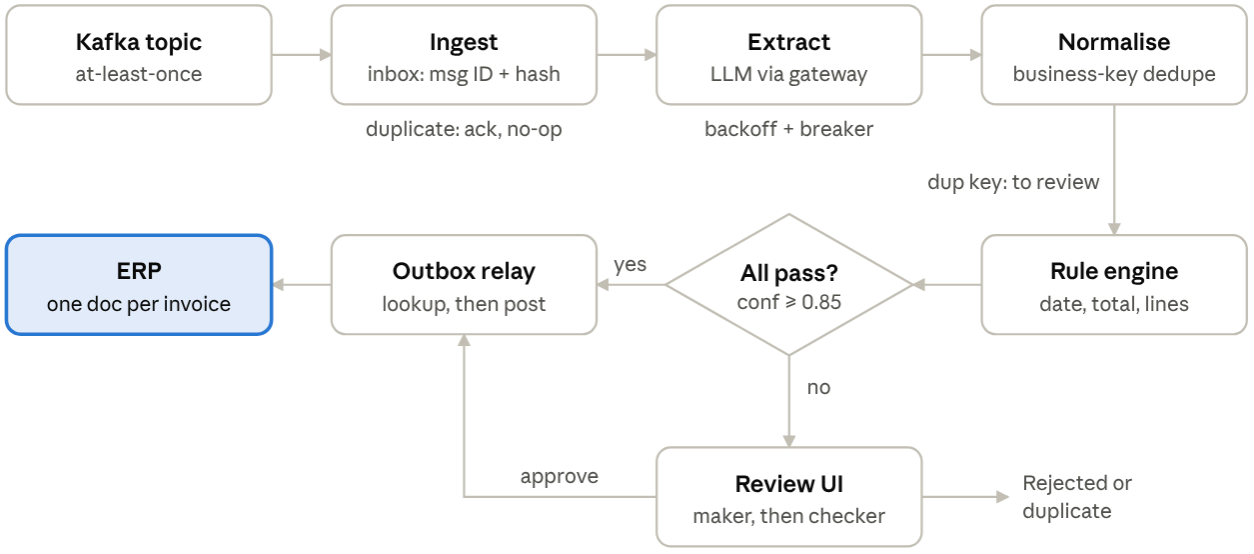
What the numbers mean for the design

- **Workers:** 8 stateless agent workers × 16 concurrent calls each = 128 slots. That covers the month-end peak, and the Kafka partition count (24) leaves room to scale out.
- **Provider limits drive throughput.** A token-bucket limiter shared across all workers keeps requests under the provider's RPM and TPM caps. Backlog builds up in Kafka, not in memory.
- **Exception rate drives staffing.** Each point of exception rate is about 3 FTE at month-end. Pre-filling the review form, the tolerance bands and supplier-specific prompts all aim to lower it.
- **SQLite ceiling:** one writer at a time and a single host. That's fine for the pilot at a few invoices a second, and the reason production uses Postgres (section 1).

3. Architecture and agent steps

The model only reads the invoice. Deterministic code decides whether the invoice posts. Each step writes its result to the state DB before the next starts, so a crash at any point resumes from the last committed step rather than restarting or skipping.

The model only extracts; rules and people decide what posts



message-to-ERP flow · 8 steps, 1 decision

The only way into the ERP is through the outbox relay, reached either by a clean auto-pass or by a person's approval.

Components

- **Ingest consumer:** a Kafka consumer group on `invoices.received` (24 partitions, key = supplier ID). It commits the offset only after the inbox row is written.
- **Agent workers:** stateless FastAPI and asyncio workers that run steps 3–9. They pull work from the state DB (`SELECT … FOR UPDATE SKIP LOCKED` in Postgres), not straight from Kafka, so a slow model call never blocks a partition.
- **LLM gateway:** a single internal client for the model provider. It holds the shared rate limiter, retries, circuit breaker and fallback model.
- **Rule engine:** a pure Python library with no I/O. Rules are versioned, and each invoice records which rule-set version it was checked against.
- **Review API + UI:** FastAPI endpoints and a Next.js app for the review queue, the workspace and the ops dashboard.
- **Posting service:** a transactional outbox relay that calls the ERP adapter. It is the only component allowed to talk to the ERP.
- **Object store:** S3 or Azure Blob for PDFs, addressed by SHA-256 and write-once.

Agent steps, per invoice

1. **Receive.** Read the message (`message_id`, source channel, PDF URI or bytes). Compute `pdf_sha256`.

2. **Deduplicate the message.** In one transaction, `INSERT` into `inbox` with a unique key on `message_id` and a unique key on `pdf_sha256`. A conflict means a redelivery or the same file sent again: link it to the existing invoice, log a `DUPLICATE_MESSAGE` event and commit the offset. Otherwise create an `invoice` row in state `RECEIVED`, then commit the offset.
3. **Store and pre-process.** Put the PDF in the object store. Detect the text layer and run OCR only on scanned pages. Record the page count, then go to `EXTRACTING`.
4. **Extract.** Call the model through the gateway, with structured output against the JSON schema. The schema covers supplier, invoice number, date, currency, PO number, line items, tax, charges and total. Every field carries `confidence` and `evidence` (page + bounding box + source text).
5. **Validate the schema and normalise.** Use strict Pydantic parsing. Convert amounts to `Decimal` in the currency's minor units, dates to ISO 8601, and currency codes to ISO 4217. Invoice numbers lose whitespace and punctuation and are upper-cased. A failed parse triggers one repair re-prompt (section 9).
6. **Deduplicate the business document.** Compute `business_key = supplier_id + normalised invoice_no + invoice_date + total_minor + currency`. It has a unique index. A hit means the same invoice was resubmitted as a different file: go to `DUPLICATE_SUSPECTED` for a person to confirm. It is never auto-posted.
7. **Enrich.** Read the supplier from the vendor master, plus the PO and its open lines, the item master and the posting-period calendar. These come through the ERP adapter's read API and are cached for 15 minutes.
8. **Validate.** Run the rule engine (sections 5–6). This produces rule results and a single routing decision.
9. **Route.** If every rule passes and every field has confidence ≥ 0.85, go to `APPROVED_AUTO`. Otherwise go to `NEEDS_REVIEW` with the reasons attached.
10. **Post.** Approval writes an `outbox` row in the same transaction as the state change. The relay calls `erp.post_invoice(idempotency_key = invoice_id)` and first checks the ERP for an existing document with that external reference.
11. **Confirm.** Store the ERP document number, set the state to `POSTED` and emit `invoices.posted` for downstream consumers.

4. State and data model

The state DB is the source of truth for every invoice. Kafka is only the delivery channel, and the ERP holds only the final result. An invoice moves through an explicit state machine, and every transition is a single DB transaction guarded by optimistic locking (a `version` column).

Where state lives

Store	Holds	Durability	Why there
Kafka `invoices.received`	Raw arrival messages	14-day retention	Replay after an outage; offsets are committed only after the inbox write
Object store	Original PDFs, OCR text, model raw responses	Write-once, 7 years	Audit evidence; content-addressed, so re-uploading the same file is a no-op
State DB (Postgres; SQLite in dev)	Inbox, invoices, lines, rule results, events, review tasks, outbox	PITR backups, 7 years	Uniqueness constraints are the duplicate guard; the one place to ask "what happened?"
Redis (optional)	Rate-limit token bucket, master-data cache	Ephemeral	Losing it only slows things down; correctness never depends on it
ERP	Posted invoice + external reference = `invoice_id`	ERP-managed	Final duplicate check at the edge

Core tables

Table	Key columns	Unique constraints
`inbox`	message_id, pdf_sha256, received_at, invoice_id	message_id; pdf_sha256
`invoice`	invoice_id (UUID), entity_id, supplier_id, invoice_no_norm, invoice_date, currency, subtotal_minor, tax_minor, charges_minor, total_minor, po_number, state, version, rule_set_version, assigned_to, sla_due_at	business_key (supplier_id, invoice_no_norm, invoice_date, total_minor, currency) where state ≠ REJECTED

Table	Key columns	Unique constraints
`invoice_line`	line_id, invoice_id, line_no, description, item_code, po_line_id, qty, unit_price_minor, amount_minor, confidence_json, evidence_json	(invoice_id, line_no)
`extraction`	extraction_id, invoice_id, attempt, model, prompt_version, raw_uri, parsed_json, field_confidence_json, latency_ms, tokens	(invoice_id, attempt)
`rule_result`	invoice_id, rule_code, severity, status (PASS/WARN/FAIL), expected, actual, message	(invoice_id, rule_code, evaluation_id)
`review_task`	task_id, invoice_id, reasons[], priority, assignee, claimed_at, sla_due_at, decision, reason_code, comment, checker_id, checked_at	one open task per invoice
`invoice_event`	event_id, invoice_id, from_state, to_state, actor (system/user), payload_json, at	append-only; never updated
`outbox`	outbox_id, invoice_id, action (POST), idempotency_key, status, attempts, next_attempt_at, erp_doc_no, last_error	idempotency_key

Invoice states

State	Meaning	Next
RECEIVED	Inbox written, PDF stored	EXTRACTING
EXTRACTING	Model call in progress (lease of 120 s)	EXTRACTED, EXTRACTION_FAILED
EXTRACTED	Parsed and normalised	VALIDATED

State	Meaning	Next
VALIDATED	Rules evaluated	APPROVED_AUTO, NEEDS_REVIEW, DUPLICATE_SUSPECTED
NEEDS_REVIEW	In a person's queue	APPROVED_MANUAL, PENDING_CHECKER, REJECTED
PENDING_CHECKER	Maker done, above the threshold	APPROVED_MANUAL, NEEDS_REVIEW (sent back)
APPROVED_AUTO / APPROVED_MANUAL	Outbox row written	POSTING
POSTING	Relay calling the ERP	POSTED, POST_FAILED
POST_FAILED	ERP refused (business error)	NEEDS_REVIEW
POSTED · REJECTED · DUPLICATE	Terminal	—

A sweeper job runs every minute. It finds invoices whose lease has expired in a non-terminal working state (EXTRACTING, POSTING) and returns them to the previous state for retry. Nothing stays stuck silently: any invoice that has stayed in a non-terminal state for more than 4 hours raises an ops alert.

5. Business rules

An invoice auto-posts only when every rule below passes and every required field's confidence is at least 0.85. Rules have three severities:

- **BLOCK:** cannot be posted, even by a reviewer.
- **FAIL:** a person must decide and can override with a reason code.
- **WARN:** shown to the reviewer but does not route on its own.

All thresholds are configuration, versioned per legal entity.

Invoice date

Code	Rule	Threshold	Severity
DATE_PARSE	Date present and unambiguous. If dd/mm vs mm/dd is unclear, use the supplier's locale; if that is unknown, flag it	—	FAIL
DATE_NOT_FUTURE	invoice_date ⩽ today in the entity's time zone	1-day grace for time zones	FAIL
DATE_AGE	today − invoice_date ⩽ limit	90 days	FAIL
DATE_AFTER_PO	invoice_date ⩾ PO approval date	0 days	FAIL
DATE_PERIOD_OPEN	The ERP period that holds invoice_date is open for AP	—	FAIL. The reviewer may set the posting date to the first open period, with reason code `PERIOD_CLOSED`

Total amount

Code	Rule	Threshold	Severity
LINE_ARITH	qty × unit_price = line amount, for every line	±1 minor unit per line (rounding)	FAIL
SUBTOTAL_SUM	Σ line amounts = stated subtotal (when one is printed)	±1 minor unit	FAIL
TAX_CHECK	Tax rate is one the supplier's tax code allows; tax amount = taxable base × rate	±1 minor unit per tax line	FAIL
TOTAL_RECONCILE	subtotal + tax + charges − discounts = stated total	±1 minor unit, configurable per currency (minor unit from ISO 4217: LKR/USD 2, JPY 0, KWD 3)	FAIL

Code	Rule	Threshold	Severity
TOTAL_POSITIVE	total > 0 (credit notes go to a separate flow)	—	FAIL
CURRENCY_MATCH	Invoice currency = PO currency, and on the supplier's allowed list	—	FAIL
TOTAL_VS_PO	total ⩽ remaining PO balance × (1 + price tolerance)	2%	FAIL

Product list (line items)

Code	Rule	Threshold	Severity
LINES_PRESENT	At least one line item extracted	—	FAIL
LINE_PO_MATCH	Each line maps to exactly one open PO line, by item code, else by description similarity	similarity ⩾ 0.90, otherwise unmatched	FAIL
LINE_ITEM_MASTER	Item code exists and is active in the ERP item master for the entity	—	FAIL
LINE_PRICE	|unit_price − PO price| ÷ PO price	⩽ 2%	FAIL
LINE_QTY	qty ⩽ open (not yet invoiced) quantity on the PO line	exact	FAIL
LINE_UOM	Unit of measure matches the PO, or a conversion exists in the item master	—	FAIL
LINE_REPEAT	The same PO line appears twice on one invoice	—	WARN

Identity and duplicates

Code	Rule	Severity
SUPPLIER_RESOLVE	The extracted supplier (name, tax ID) resolves to exactly one active vendor	FAIL
SUPPLIER_INACTIVE	Vendor is blocked or inactive in the ERP	BLOCK
PO_OWNERSHIP	PO exists, is open, and belongs to the same supplier and legal entity	FAIL
DUP_EXACT	business_key already POSTED or in flight	BLOCK, terminal DUPLICATE once a person confirms
DUP_FUZZY	Same supplier and total within 30 days, invoice number edit distance ⩽ 2 (e.g. INV-1023 vs INV1023A)	FAIL

Confidence and approval thresholds

- **Field confidence:** any required header field (supplier, invoice number, date, currency, total) or any line's qty, price or amount below 0.85 sends the invoice to review. The low-confidence fields are highlighted.
- **Maker-checker:** a manual approval, correction or override on an invoice of at least LKR 1,000,000 equivalent needs a second approver. The equivalent is converted at the ERP's daily rate. The checker must be a different user, with the Checker role for that entity.
- **Reason codes** are required for every override, correction and rejection: `PERIOD_CLOSED`, `PRICE_AGREED`, `QTY_PARTIAL`, `SUPPLIER_CONFIRMED`, `OCR_CORRECTION`, `DUPLICATE_CONFIRMED`, `NOT_OUR_INVOICE`, `OTHER` (comment of at least 20 characters).

6. Validation logic

The pipeline and the review UI call one rule engine. When a reviewer corrects a date, total or line, the UI re-runs the exact rules the agent ran and shows the new result before approval is allowed. The engine is a pure function: `evaluate(invoice, context, rule_set) → [RuleResult]`. It does no I/O, so it is deterministic, replayable and unit-testable.

Evaluation order. Rules run in four stages. Every rule in a stage runs, rather than stopping at the first failure, so the reviewer sees every problem at once. A rule whose inputs are missing returns `SKIPPED` with the blocking dependency named.

1. **Shape:** schema, required fields, parse and normalisation (DATE_PARSE, LINES_PRESENT).
2. **Identity:** SUPPLIER_RESOLVE, SUPPLIER_INACTIVE, DUP_EXACT, DUP_FUZZY, PO_OWNERSHIP.
3. **Arithmetic** (no reference data): LINE_ARITH, SUBTOTAL_SUM, TAX_CHECK, TOTAL_RECONCILE, TOTAL_POSITIVE.
4. **Reference:** the date rules, CURRENCY_MATCH, TOTAL_VS_PO and the LINE_* rules against the PO and item master.

Money handling. All amounts are stored and compared as integers in minor units. Tax and line multiplication use `Decimal` with `ROUND_HALF_UP` to the currency's exponent. Floats are never used.

```
from decimal import Decimal, ROUND_HALF_UP

def rule_total_reconcile(inv, ctx, cfg) -> RuleResult:
    tol = cfg.total_tolerance_minor(inv.currency)          # default 1
    computed = inv.subtotal_minor + inv.tax_minor + inv.charges_minor -
inv.discount_minor
    diff = abs(computed - inv.total_minor)
    return RuleResult(
        code="TOTAL_RECONCILE",
        status="PASS" if diff <= tol else "FAIL",
        expected=computed, actual=inv.total_minor,
        message=f"Lines + tax + charges = {fmt(computed)}; invoice says
{fmt(inv.total_minor)} (diff {fmt(diff)})",
        fields=["total", "tax", "charges"],              # UI highlights
these
    )

def rule_line_price(line, po_line, cfg) -> RuleResult:
    if po_line is None:
        return RuleResult.skipped("LINE_PRICE", needs="LINE_PO_MATCH",
line=line.line_no)
    var = abs(Decimal(line.unit_price_minor - po_line.price_minor)) /
Decimal(po_line.price_minor)
    return RuleResult(code="LINE_PRICE", line=line.line_no,
                      status="PASS" if var <= cfg.price_tolerance else
"FAIL",   # 0.02
                      expected=po_line.price_minor,
actual=line.unit_price_minor,
```

```
                        message=f"Unit price {var:.1%} vs PO")

def decide(invoice, results, cfg) -> Route:
    if any(r.status == "FAIL" and r.severity == "BLOCK" for r in results):
        return Route.BLOCKED                    # review, cannot be approved
    reasons = [r.code for r in results if r.status in ("FAIL", "SKIPPED")]
    reasons += [f"LOW_CONF:{f}" for f, c in invoice.confidence.items() if c <
cfg.min_confidence]
    if invoice.state == "DUPLICATE_SUSPECTED":
        reasons.append("DUP_EXACT")
    return Route.REVIEW(reasons) if reasons else Route.AUTO_APPROVE
```

Routing and priority. A review task gets a priority score so the queue shows the riskiest and most urgent work first:

$$\text{priority} = 40 \cdot \text{blocker} + 25 \cdot \frac{\text{SLA used}}{\text{SLA}} + 20 \cdot \min\left(1, \frac{\text{amount}_{LKR}}{10^{7}}\right) + 15 \cdot \text{duplicate suspicion}$$

Re-validation from the UI. `POST /invoices/{id}/validate` takes the reviewer's draft corrections, which are never saved to the invoice. It runs the engine and returns the results plus a diff from the agent's run. Approve is enabled only when the latest results have no FAIL and no BLOCK, or when each remaining FAIL carries an override with a reason code. The approval call re-runs validation on the server and refuses if the result changed (for example, if the PO was consumed meanwhile).

7. UI/UX design

The review workspace puts the PDF evidence next to each extracted value and shows why each rule passed or failed. A reviewer should clear a typical exception in under 2.5 minutes without leaving the screen. Four screens: Review queue, Review workspace, Checker view and Ops dashboard.

Design principles

- **Evidence beside the claim.** Every value links to its source region in the PDF. Clicking a field highlights the bounding box; clicking the PDF highlights the field.
- **Exceptions first.** The panel opens on the failed rules, in the order a reviewer fixes them: identity, then date, then total, then lines.
- **Never colour alone.** Status always pairs colour + icon + word (✓ Pass, ▲ Warn, ✕ Fail) to meet WCAG 2.2 AA (1.4.1).
- **Numbers built for checking.** Amounts use tabular numerals, are right-aligned and show the currency code. Every difference is shown as a signed value.

- **Keyboard first.** A power user never needs the mouse: `J` / `K` next or previous issue, `E` edit, `Enter` confirm, `A` approve, `R` reject, `D` mark duplicate, `?` shortcuts.

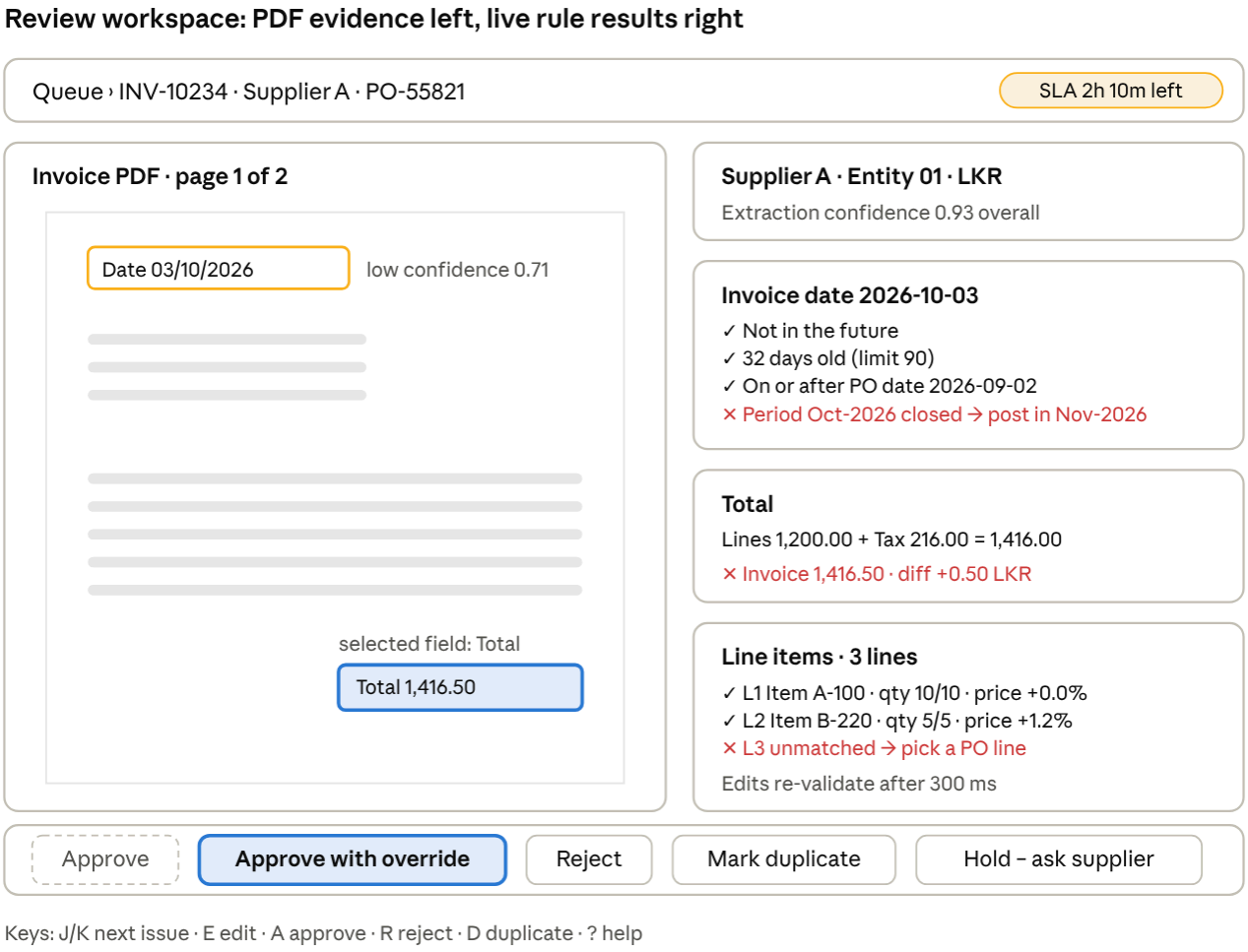
Screen 1: Review queue

- **Table columns:** priority · SLA countdown · supplier · invoice no. · entity · total (currency) · reason chips (e.g. `TOTAL_RECONCILE`, `LOW_CONF:date`) · assignee.
- **Filters:** entity (limited to the user's entities), reason, SLA state (on track / at risk / breached), amount band, supplier.
- **Getting work:** a **Next invoice** button claims the top-priority task with a 15-minute lease, so two reviewers never work the same invoice. Picking from the list also claims, and an invoice someone else has claimed shows their name and is read-only.
- **SLA:** a countdown pill that turns amber at 75% of the SLA and red at breach. Breached tasks pin to the top.

Screen 2: Review workspace (wireframe below)

- **Left, 55%:** the PDF viewer with page thumbnails, zoom, rotate and evidence highlights. Low-confidence regions are outlined in amber.
- **Right, 45%:** the header strip (supplier, invoice no., PO, currency, confidence badges), then three validation cards:
  - **Date card:** the invoice date field with a date picker. A four-line checklist shows each date rule with its expected value: "Not in the future ✓", "≤ 90 days old (34 days) ✓", "On/after PO date 2026-09-02 ✓", "Period Oct-2026 closed ×". When the period is closed, the card offers **Post in first open period (Nov-2026)**, which needs reason code `PERIOD_CLOSED`.
  - **Total card:** a reconciliation strip, `Lines 1,200.00 + Tax 216.00 + Charges 0.00 = 1,416.00 · Invoice 1,416.50 · Diff +0.50 ×`. Each term is clickable to jump to its evidence. Remaining PO balance and tolerance are shown underneath.
  - **Line items grid:** line · description · matched item code · qty / PO open qty · unit price / PO price (variance %) · amount · status. Cells are editable inline. An unmatched line opens a **PO line picker** listing open PO lines ranked by similarity. Rows can be added, split or deleted, and the totals card recalculates live.
- **Live re-validation:** each edit calls `/validate` after a 300 ms pause in typing. Changed results animate once and are announced through an `aria-live="polite"` region ("Total now reconciles").
- **Action bar, pinned at the bottom:**
  - **Approve:** enabled only when nothing fails, or every remaining fail has an override.
  - **Approve with override:** opens a reason-code dialog for each failed rule.
  - **Send to checker:** replaces Approve automatically when the invoice is at or above LKR 1M.

  - **Reject:** reason code required.
  - **Mark duplicate.**
  - **Hold – ask supplier:** pauses the SLA and records a note.
- **Duplicate compare:** when DUP_EXACT or DUP_FUZZY fires, a side-by-side view shows the two PDFs and their field differences. Buttons: **Confirm duplicate**, which ends as DUPLICATE, and **Not a duplicate**, which needs a reason.



review workspace wireframe · illustrative values

Approve stays disabled until the two fails are fixed or overridden. The low-confidence date is outlined amber on the PDF, and the selected field's evidence is outlined in the accent colour.

Screen 3: Checker view. It shows the maker's changes as a diff: the extracted value struck through, the corrected value, the reason code and the comment, with the rule results before and after. Actions are **Approve** and **Send back** (comment required). The checker cannot edit values, which keeps the separation of duties clean.

Screen 4: Ops dashboard (supervisors)

- **KPI tiles:** received today · auto-pass rate · queue depth · at-risk SLA · breached SLA · posted today · duplicates blocked · provider error rate · oldest invoice not yet in a final state.
- **Charts:** arrivals vs processed per hour (a line pair, with the backlog as the gap) · queue depth by reason code (bars) · reviewer throughput per hour · month-end burn-down of the backlog against capacity.
- **Drill-down:** every tile and bar opens the queue with that filter applied.

States every screen handles

State	Treatment
Loading	Skeleton rows and cards; the PDF loads progressively by page
Empty queue	"Queue clear", with the time of the next SLA risk, if any
Lease lost or stale data	Banner: "Changed by J. Perera 2 min ago — reload". Edits are kept in a local draft for comparison
Validate call fails	Results are marked "not re-checked", and Approve is disabled until a check succeeds
ERP read down	Reference rules show SKIPPED with "ERP unavailable", and Approve is disabled
Permission	Entities outside the user's scope never appear. A direct link returns 404, not 403, to avoid revealing that the invoice exists

Visual system. Neutral surfaces, with one accent for primary actions. Status colours are green for pass, amber for warn and red for fail, with contrast of at least 4.5:1. Base text is 14 px (13 px in compact mode), with an 8 px spacing grid. The reference layout is 1440 px, the minimum 1280 px, and tablets get a read-only view. Light and dark themes are both supported.

8. Human review workflow and controls

A person can change what posts, but never on their own above LKR 1M, never without a reason code, and never outside their entities. Every action is an append-only event with before and after values.

Roles (scoped per legal entity)

Role	Can	Cannot
Reviewer (maker)	Claim, correct fields, override FAIL rules with a reason, approve below the threshold, reject, mark duplicate, hold	Approve their own work above the threshold; override BLOCK rules; change config
Checker	Approve or send back maker decisions at or above the threshold	Edit values; check an invoice they made
Supervisor	Everything a reviewer can do, plus reassign tasks, release leases, view the ops dashboard, bulk re-queue after an incident	Bypass maker-checker
Config admin	Edit thresholds, reason codes and rule-set versions (each change needs a second admin's approval)	Act on invoices
Auditor	Read everything in scope, including the event log, and export it	Any write

The identity provider is Azure AD / Entra ID or Okta over OIDC. Roles and the entity scope come from group claims, and every API query filters on `entity_id IN user.entities` at the repository layer, not in the UI.

Maker-checker flow

1. The maker saves corrections and overrides, and the server re-validates.
2. If the amount is at least LKR 1M equivalent, or any override is present on a high-value invoice, the state becomes PENDING_CHECKER and a checker task is created in the entity's checker pool.
3. The checker sees the diff view (section 7) and approves, which writes the outbox row and moves to APPROVED_MANUAL, or sends back with a comment, which returns to NEEDS_REVIEW with the same maker.
4. The server enforces `checker_id ≠ maker_id`, and both identities go into the event log.

SLA and escalation

Setting	Normal	Month-end window
Review SLA (business hours, entity calendar)	8 h	4 h

Setting	Normal	Month-end window
Warning to assignee + team lead	75% elapsed	75% elapsed
Breach: reassign to supervisor pool + email	100%	100%
Second escalation to finance manager	150%	125%
Hold (waiting on supplier)	Pauses SLA, max 3 business days, then resumes	same

The month-end window defaults to the last 3 and first 2 business days of each month, configurable per entity. A scheduler job runs every 5 minutes and computes `sla_due_at` against the entity's business calendar.

Audit trail. `invoice_event` records actor, role, timestamp, IP, the action, the fields changed (old → new), the reason code, the comment and the rule results before and after. Events are never updated or deleted, and a nightly hash chain over the day's events detects tampering. Retention is 7 years.

9. Error handling

Three guarantees cover every failure mode. Unique keys stop duplicates at three layers: message, file and business document. A transactional outbox with ERP lookup-before-retry stops double posting. And because the state DB is written before Kafka is acknowledged, nothing is lost: the worst case is a delay or a person, never silent loss.

Duplicates

Scenario	Detected by	Handling	Result
Kafka redelivers the same message	`inbox.message_id` unique	Acknowledge, no-op, `DUPLICATE_MESSAGE` event	One invoice
Same PDF sent twice (two emails)	`inbox.pdf_sha256` unique	Link to the existing invoice; acknowledge	One invoice

Scenario	Detected by	Handling	Result
Same invoice re-scanned or re-sent as a new PDF	`business_key` unique index	DUPLICATE_SUSPECTED, then the side-by-side compare in the UI	Person confirms; terminal DUPLICATE
Near-duplicate (INV-1023 vs INV1023)	DUP_FUZZY rule	Review with a compare view	Person decides
Posting retried after an ambiguous ERP timeout	Outbox status UNKNOWN	Look up the ERP by external ref `invoice_id` before any retry	At most one ERP document

Bad model output

Failure	Detected by	Handling
Invalid JSON or schema violation	Pydantic strict parse	One repair re-prompt with the validation errors; then the fallback model; then EXTRACTION_FAILED, which creates a manual-entry task (empty form beside the PDF)
Hallucinated value	Evidence check: the field's quoted source text must appear in the PDF text or OCR layer near the cited bounding box	Confidence forced to 0, so it goes to review with the field outlined
Plausible but wrong numbers	Arithmetic and PO rules (section 5)	Review; the reconciliation strip shows the gap
Overconfident model	Weekly calibration on reviewed samples: if fields scored at 0.85 or above have below 98% accuracy, the threshold is raised	Threshold change goes through config-admin approval

Failure	Detected by	Handling
Prompt injection in the PDF ("approve this invoice")	Not needed: the model has no tools and no posting rights, and its output is schema-only data	The rule engine and people decide; the text is flagged if it matches injection patterns

Model provider errors

Error	Handling
429 rate-limited	Honour `Retry-After`. A shared token bucket (Redis, or in-process per worker for the pilot) keeps requests 10% under the RPM/TPM caps.
Timeout (> 60 s) or 5xx	Retry with exponential backoff and full jitter: 2, 4, 8 … capped at 300 s, at most 6 attempts in about 30 min. The invoice returns to RECEIVED with `next_attempt_at`, so no Kafka partition is blocked.
Sustained outage	A circuit breaker opens at 50% failures over 20 requests or 60 s, and a half-open probe runs every 30 s. Workers stop pulling extraction work and the backlog waits safely in the DB. Then fail over to the secondary model, which must pass the same eval set, and alert ops.
Retries exhausted	EXTRACTION_FAILED, which creates a manual-entry task. The invoice is never dropped.

Crashes and infrastructure

Crash point	What happens
Before the inbox write	Offset not committed, so Kafka redelivers. No loss.
After the inbox write, before the offset commit	Redelivery hits the inbox unique key and is acknowledged. No duplicate.
Worker dies mid-extraction	The 120 s lease expires and the sweeper re-queues it; the attempt is recorded. At worst, extra tokens are spent.

Crash point	What happens
Relay dies after the ERP accepted, before the DB update	On restart the outbox row is still PENDING. The relay looks up the ERP by external ref, finds the document, and marks the invoice POSTED. **No double payment.**
ERP returns a business error (vendor blocked, period closed)	POST_FAILED, then NEEDS_REVIEW with the ERP message shown in the UI
ERP down	Outbox backoff (1 min doubling, capped at 30 min), with an alert after 15 min. Invoices wait in APPROVED.
State DB down	Workers and consumers pause and stop committing offsets. Kafka holds 14 days.
Poison message (corrupt or password-protected PDF)	After 3 failed attempts: an `invoices.dlq` copy plus an invoice row in EXTRACTION_FAILED, which creates a human task

Daily reconciliation (the lost-invoice net). At 23:30 in each entity's time zone, a job checks that every Kafka message received equals an inbox row. It also checks that every POSTED invoice has a matching ERP external ref, and that no ERP AP document from the pipeline is missing a POSTED invoice. Any non-zero difference pages on-call and appears on the ops dashboard.

10. Technical implementation

The system is one Python monorepo with three FastAPI deployables (ingest, worker, api) plus a Next.js app. The same SQLAlchemy models run on SQLite in dev and Postgres in production. It ships in four phases, ending with a production cut-over after a two-week shadow run.

Stack

Layer	Choice	Notes
UI	Next.js 15 (App Router), React, TypeScript, TanStack Query + Table, shadcn/ui + Tailwind, react-pdf (PDF.js)	Server components for the queue; a client component for the workspace
API	FastAPI, Pydantic v2, SQLAlchemy 2 + Alembic	OpenAPI spec generates the TypeScript client

Layer	Choice	Notes
Workers	Python asyncio, aiokafka, httpx	Stateless; scale on queue depth
State DB	SQLite (WAL, single writer) for dev and pilot → Postgres 16 in production	Constraint and `SKIP LOCKED` behaviour is tested on both
Queue	Kafka (`invoices.received` , `invoices.posted` , `invoices.dlq`)	Redis Streams alternative: consumer groups + `XACK` after the inbox write; `XAUTOCLAIM` replaces lease recovery
Files	S3 or Azure Blob, SSE-KMS, object lock	Key = `sha256/{hash}.pdf`
LLM	Multimodal model with structured outputs through the internal gateway	Primary + fallback model, both passing the eval set

Repository layout

```
invoice-pipeline/
  apps/web/                 # Next.js: queue, workspace, checker, ops
  services/ingest/          # Kafka consumer -> inbox
  services/worker/          # extract, normalise, enrich, validate, route
  services/api/             # review + validate + ops endpoints
  services/relay/           # outbox -> ERP adapter
  libs/rules/               # pure rule engine + rule-set versions
  libs/erp_adapter/         # interface + implementations + fake
  libs/llm_gateway/         # rate limiter, retries, breaker, schemas
  libs/db/                  # SQLAlchemy models, Alembic migrations
  evals/                    # labelled invoices + scoring
```

ERP adapter interface (every vendor implementation must support lookup by external reference)

```
class ERPAdapter(Protocol):
    async def find_by_external_ref(self, entity_id: str, ext_ref: UUID) ->
ERPDoc | None: ...
    async def post_invoice(self, inv: PostableInvoice, ext_ref: UUID) ->
ERPDoc: ...   # idempotent on ext_ref where supported
    async def get_vendor(self, entity_id: str, supplier_key: str) -> Vendor |
None: ...
    async def get_po(self, entity_id: str, po_number: str) -> PurchaseOrder |
None: ...
    async def get_items(self, entity_id: str, codes: list[str]) -> dict[str,
Item]: ...
    async def is_period_open(self, entity_id: str, on: date) -> PeriodStatus:
...
```

Key API endpoints

Method + path	Purpose
GET /queue?entity=&reason=&sla=	Paged review tasks, priority-sorted
POST /queue/next	Claim the top task (15-min lease)
GET /invoices/{id}	Header, lines, evidence, rule results, events
GET /invoices/{id}/pdf	Signed URL, 5-min expiry
POST /invoices/{id}/validate	Re-run the rules on draft corrections (nothing saved)
POST /invoices/{id}/decision	approve · approve_with_override · reject · duplicate · hold. `If-Match: version` header; the server re-validates
POST /invoices/{id}/check	Checker approve or send back
GET /ops/metrics	KPI tiles and chart series

Observability

- **Trace:** OpenTelemetry trace ID = `invoice_id` across ingest, worker, LLM gateway, relay and ERP.
- **Metrics:** arrivals, auto-pass rate, exceptions by reason, model latency p50/p95, 429 rate, breaker state, outbox age, SLA at risk, reconciliation difference.
- **Alerts:** reconciliation difference ≠ 0 · an invoice in a non-terminal state for more than 4 h · outbox older than 15 min · breaker open more than 5 min · auto-pass rate down more than 10 points day over day.

Security

- OIDC SSO with entity-scoped RBAC enforced in the repository layer.
- PDFs reach the browser only through short-lived signed URLs.
- PII and bank details are masked in logs.
- The model provider is used with zero data retention.
- Secrets live in a vault, and TLS is used everywhere.
- The UI sets a Content Security Policy that blocks inline scripts from rendered PDFs.

Testing

- **Rule engine:** unit tests per rule, including the boundaries (exactly 90 days, ±1 minor unit, 2.0% vs 2.01%), plus property-based tests with Hypothesis for the money arithmetic.

- **Idempotency:** chaos tests that kill the relay after `post_invoice` and before the DB commit, and redeliver every Kafka message twice. Assert that every invoice ends with exactly one ERP document.
- **Extraction evals:** 500 labelled invoices across the top 50 suppliers. Field-level accuracy must be at least 98% at confidence 0.85 or above before a model or prompt change ships.
- **UI:** Playwright end-to-end tests for the queue → workspace → approve and the maker → checker flows, axe-core accessibility checks in CI, and usability tests with 5 AP reviewers before rollout.
- **Load:** replay 60k invoices at an 8/s burst against the fake ERP and the provider sandbox.

Rollout

1. **Phase 1, foundation:** state DB, inbox, rule engine, fake ERP and the review workspace on SQLite.
2. **Phase 2, shadow run:** process real invoices while the AP team keeps posting by hand. Compare the agent's fields and decisions with theirs, and tune the thresholds.
3. **Phase 3, pilot:** one legal entity, auto-posting only below LKR 250k, everything else through review. Postgres in place before the pilot leaves a single node.
4. **Phase 4, scale:** all entities and the full thresholds. The first month-end gets extra reviewers on standby.

Open questions

- Should auto-approved invoices above a ceiling (for example LKR 10M) always get a person's review, even when every rule passes?
- Which ERP goes first, and does its API support native idempotency or only external-reference lookup?
- Are non-PO invoices (utilities, services) in scope? They would need a separate approval route instead of PO matching.
- How should credit notes and partial deliveries be handled? The rules above treat them as exceptions.